CHƯƠNG 4:
TÍNH TOÁN BẤT ĐỒNG BỘ

Học phần: Tính toán song song và phân tán

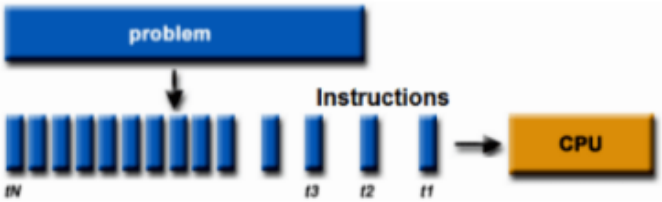
Phần I: Các mô hình tính toán nền tảng (Tuần tự, Đồng thời, Song song, Bất đồng bộ)

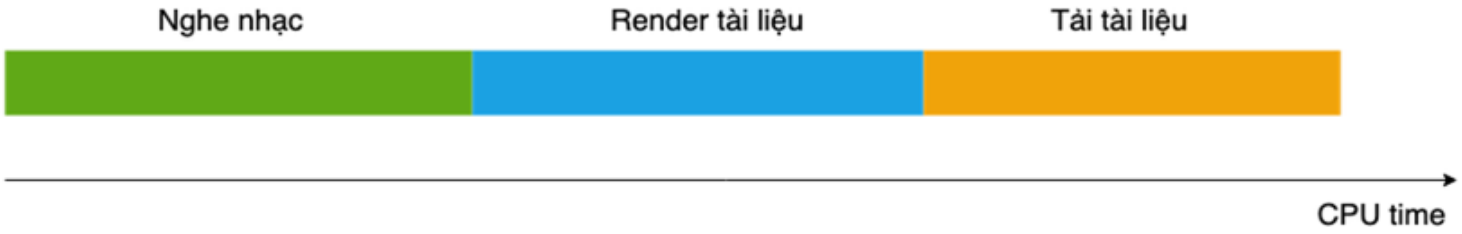
GIỚI THIỆU

➢ Xử lý tuần tự

□ Xử lý 1 tác vụ (task) trong một khoảng thời gian;

□ Các task sẽ được thực thi theo thứ tự.





※ **Phân tích học thuật:** Tổng thời gian thực thi T_total = t1 + t2 + t3. CPU bị lãng phí hoàn toàn (idle waiting) khi task chờ đợi I/O.

GIỚI THIỆU

➢ Xử lý đồng thời

□ HĐH quản lý chương trình, mỗi chương trình ~ process (tiến trình) → cấp process id (PID) để HĐH quản lý.

□ CPU core chia các task của process thành các task nhỏ hơn + sắp xếp xen kẽ nhau → tận dụng thời gian rảnh của task này đi thực hiện task khác.

□ → Máy tính xử lý nhiều việc cùng lúc tại cùng thời điểm, tuy nhiên, bên dưới CPU core chỉ có thể thực thi một task nhỏ trong task lớn tại thời điểm đó.



※ **Cơ chế vi mô:** Phân chia thời gian (Time-Slicing) và chuyển đổi ngữ cảnh (Context Switching) tạo ảo giác chạy song song trên 1 Core.

GIỚI THIỆU

➢ Xử lý đồng thời

□ Phân chia, điều phối nhiều tác vụ (task) khác nhau trong cùng một khoảng thời gian.

□ Tại 1 thời điểm, chỉ xử lý 1 task.



※ **Bản chất cốt lõi:** Đồng thời (Concurrency) nói về CẤU TRÚC chương trình (nhiều việc cùng tiến triển), không đồng nghĩa với song song vật lý.

GIỚI THIỆU

➢ Xử lý song song

- □ Nhiều task khác nhau được xử lý trong cùng 1 thời điểm;
- □ Các task hoàn toàn độc lập nhau.
- □ Chỉ có thể thực hiện trên máy có số nhân (core) lớn hơn 1.



※ **Điều kiện phần cứng:** Bắt buộc kiến trúc Đa nhân (Multi-core / Multi-CPU). Nhiều lệnh được thực thi vật lý đồng thời tại cùng tích tắc.

GIỚI THIỆU

➢ Xử lý song song

□ Trên thực tế, trên mỗi nhân của CPU vẫn xảy ra quá trình xử lý đồng thời, với điều kiện là tại một thời điểm không xảy ra việc xử lý cùng một task trên hai nhân CPU khác nhau.



※ **Kiến trúc phân cấp:** Hệ điều hành hiện đại kết hợp: Song song thực sự giữa các CPU Core và Đồng thời chia sẻ thời gian bên trong từng Core.

GIỚI THIỆU

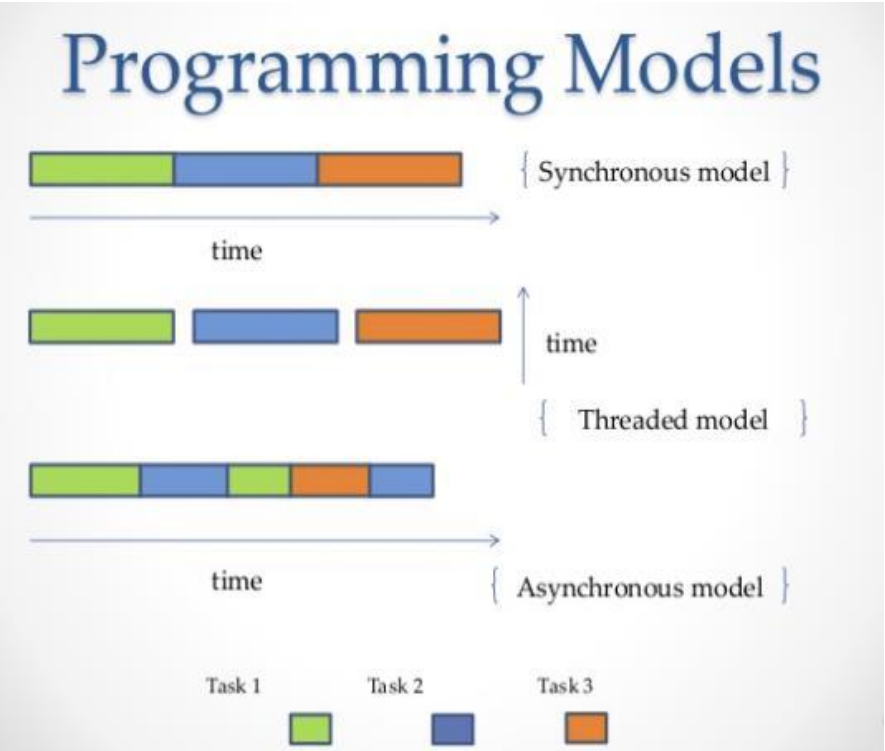
➢ Bất đồng bộ

□ Cho phép các task được xử lý không theo thứ tự (có thể chuyển sang task khác trước khi task trước đó hoàn thành).

□ Có thể giải quyết nhiều yêu cầu đồng thời.

□ Hoàn thành trong khoảng thời gian ngắn hơn.

□ **VD: Đi thi, bãi giữ xe, ...**



※ **So sánh 3 mô hình lập trình:** Synchronous (chặn theo thứ tự) vs Threaded (mỗi task 1 luồng tốn RAM) vs Asynchronous (1 luồng không chặn, xen kẽ linh hoạt).

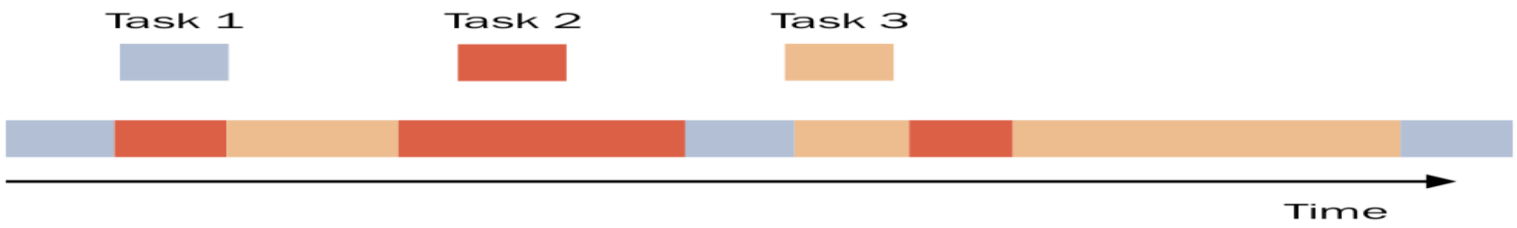
GIỚI THIỆU

➢ Mô hình bất đồng bộ thực thi đồng thời

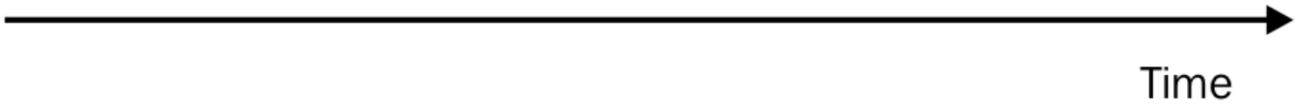
□ Trong một khoảng thời gian, các task thực hiện xen kẽ nhau;

□ Nằm trong một luồng điều khiển duy nhất → khi 1 task đang thực thi thì các task khác không thực thi (việc thực thi của 1 task có thể bị tạm dừng và sau đó tiếp tục lại);

□ Sử dụng một luồng điều khiển chính duy nhất, cả trong hệ thống đơn bộ xử lý (uniprocessor systems) và hệ thống đa bộ xử lý (multiprocessor systems).



※ **Nguyên lý Single-Thread Cooperative:** Nền tảng của Event Loop trong Python asyncio: Task tạm dừng (yield) khi chờ I/O, nhường quyền cho task khác chạy trên cùng 1 luồng.



GIỚI THIỆU

➢ Phân tích bản chất qua 2 ví dụ đời sống gốc

🅿️ VÍ DỤ 1: BÃI GIỮ XE THÔNG MINH

• Tuần tự (Blocking): Bảo vệ phát thẻ xe cho Xe 1 → Đứng chờ Xe 1 chạy vào bãi tìm chỗ đỗ xong xuôi → Mới phát thẻ cho Xe 2.

→ Hậu quả: Cổng giữ xe bị ùn tắc nghiêm trọng! Thời gian chờ là tổng thời gian tìm chỗ đỗ của từng xe.

• Bất đồng bộ (Async Non-blocking): Bảo vệ quẹt thẻ trao vé trong 2 giây → Xe 1 tự vào bãi tìm chỗ (I/O chờ) → Bảo vệ đón ngay Xe 2!

→ Bản chất: Luồng bảo vệ không bị chặn (Non-blocking). Nhiều người tìm chỗ đỗ đồng thời, cổng ra vào thông suốt liên tục.

📝 VÍ DỤ 2: ĐI THI (LÀM BÀI TRẮC NGHIỆM)

• Tuần tự (Blocking): Thí sinh làm từ Câu 1 → Câu 2 → Gặp Câu 3 quá khó → Ngồi nghĩ 30 phút → Hết giờ mà chưa kịp đọc các câu sau.

→ Hậu quả: Bị nghẽn toàn bộ bài thi! Điểm số thấp vì thời gian bị chôn chân tại câu khó.

• Bất đồng bộ (Async): Gặp Câu 3 khó (chờ xử lý) → Đánh dấu tạm gác lại (Yield/Pause) → Chuyển sang làm ngay Câu 4, 5, 6 dễ!

→ Bản chất: Không để thời gian chờ làm tắc nghẽn công việc. Khi làm hết câu dễ, quay lại tiếp tục giải câu 3 (Resume).

※ Bài học cốt lõi: Bất đồng bộ không làm cho từng phép tính nhanh hơn, mà giúp HỆ THỐNG KHÔNG BỊ TREO khi có tác vụ mất thời gian chờ đợi.

GIỚI THIỆU

➢ Ma trận đối chiếu 4 mô hình tính toán nền tảng

Tiêu chí đối chiếu	Tuần tự (Sequential)	Đồng thời (Concurrency)	Song song (Parallelism)	Bất đồng bộ (Asynchronous)
Đơn vị điều phối	Chỉ thị lệnh (Instruction)	Process / Thread	CPU Core vật lý	Coroutine / Task
Yêu cầu phần cứng	≥ 1 Core bất kỳ	≥ 1 Core (Time-slicing)	Đa nhân (Multi-core > 1)	≥ 1 Core (Single-thread)
Khi gặp I/O chờ	Nghẽn hoàn toàn (Block)	HĐH chuyển ngữ cảnh	Core khác vẫn chạy	Tạm nhường quyền (Yield)
Phù hợp bài toán	Tuần tự, đơn giản	I/O blocking đa luồng	CPU-bound nặng	I/O-bound nhiều kết nối

※ **Cầu nối học thuật:** Nền tảng này mở đường trực tiếp sang Phần II: concurrent.futures (ThreadPool/ProcessPool) và Phần IV: asyncio trong Python.